

LAPORAN TUGAS
MAIN MEMORY



Dosen Pengampu : Triawan Adi Cahyanto, M.K

Disusun Oleh:

Muhammad maulana wiramatajaya 2410651006

PROGRAM STUDY TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2024

DAFTAR ISI

BAB 1	1
PENDAHULUAN	1
1.1. Latar Belakang	1
1.2. Tujuan Tujuan Praktikum.....	1
1.3. Spesifikasi Sistem	1
BAB 2.....	3
DASAR TEORI.....	3
2.1. Main Memory	3
2.2. Contiguous Memory Allocation.....	3
2.3. Paging	3
2.4. Page Table Structures.....	3
2.5. Swapping.....	3
2.6. Example Intel 32 & 64-Bit Architectures	3
2.7. Example ARMv8 Architecture.....	3
BAB 3.....	4
PRAKTIKUM	4
3.1. DESKRIPSI TUGAS	4
3.2. SOURCE CODE.....	4
3.3. Screenshot output.....	14
3.4. Pejelasan	14
BAB 4.....	15
ANALISIS DAN PEMBAHASAN	15
BAB 5.....	16
DAFTAR PUSTAKA	16

BAB 1

PENDAHULUAN

1.1. Latar Belakang

1. Menganalisis kebutuhan fungsional dan non-fungsional dari sistem E-commerce yang akan dikembangkan.
2. Merancang arsitektur sistem dan user interface (UI) aplikasi E-commerce dengan pendekatan User-Centered Design (UCD).
3. Mengimplementasikan dan menguji aplikasi E-commerce berdasarkan framework Scrum pada metode Agile.

1.2. Tujuan Tujuan Praktikum

1. Memahami konsep dasar manajemen memori dalam sistem operasi
2. Mengimplementasikan algoritma alokasi memori berurutan (contiguous)
3. Memahami dan mensimulasikan mekanisme paging
4. Mengimplementasikan struktur page table
5. Memahami konsep swapping dan implementasinya
6. Menganalisis arsitektur memori Intel dan ARM

1.3. Spesifikasi Sistem

Berdasarkan tugas analisis *memory layout* yang sedang Anda kerjakan, berikut adalah spesifikasi sistem dan lingkungan pengembangan yang ideal serta teknis untuk menjalankan kode C tersebut agar hasilnya akurat:

1. Lingkungan Pengembangan (Software)

Sistem Operasi: Disarankan menggunakan keluarga Unix-like (Linux atau macOS) karena pengelolaan memorinya lebih transparan untuk dipelajari melalui fungsi-fungsi standar. Jika di Windows, gunakan WSL (Windows Subsystem for Linux) atau MinGW.

2. Konfigurasi Arsitektur

Memori akan ditampilkan dalam format heksadesimal. Perbedaan arsitektur akan mempengaruhi panjang alamat:

- x86_64 (64-bit): Alamat memori biasanya sepanjang 12-16 karakter heksadesimal (contoh: 0x7ffce234b8ac).

- x86 (32-bit): Alamat memori lebih pendek, biasanya 8 karakter heksadesimal (contoh: 0x08048000).

3. Pemahaman Visual Layout Memori

Sistem operasi modern mengatur memori dalam hierarki tertentu. Saat Anda menjalankan kode tersebut, Anda akan melihat urutan alamat dari yang terendah ke tertinggi (atau sebaliknya):

BAB 2

DASAR TEORI

2.1. Main Memory

Memori utama (main memory) adalah tempat penyimpanan yang dapat diakses langsung oleh CPU. Dalam

- sistem operasi, manajemen memori meliputi:
- Address Binding: Proses pemetaan alamat logis ke alamat fisik
- Logical vs Physical Address: Alamat yang digunakan program vs alamat sebenarnya di RAM
- Memory Protection: Melindungi area memori antar proses

2.2. Contiguous Memory Allocation

Alokasi memori berurutan membagi memori menjadi beberapa partisi. Ada tiga algoritma utama:

- First-Fit: Alokasi pada hole pertama yang cukup besar
- Best-Fit: Alokasi pada hole terkecil yang cukup besar
- Worst-Fit: Alokasi pada hole terbesar

2.3. Paging

2.4. Page Table Structures

2.5. Swapping

2.6. Example Intel 32 & 64-Bit Architectures

2.7. Example ARMv8 Architecture

BAB 3

PRAKTIKUM

3.1. DESKRIPSI TUGAS

3.2. SOURCE CODE

1. Main Memory

```
#include <stdio.h>
#include <stdlib.h>
int global_var = 100; // Data segment
int uninitialized_var; // BSS segment
void print_addresses() {
    int local_var = 50; // Stack
    int *heap_var = (int*)malloc(sizeof(int)); // Heap
    *heap_var = 75;
    printf("=== MEMORY ADDRESS LAYOUT ===\n\n");
    printf("1. Text Segment (Code):\n");
    printf(" Function address: %p\n\n", (void*)print_addresses);
    printf("2. Data Segment (Initialized Global):\n");
    printf(" global_var address: %p, value: %d\n\n",
        (void*)&global_var, global_var);
    printf("3. BSS Segment (Uninitialized Global):\n");
    printf(" uninitialized_var address: %p, value: %d\n\n",
        (void*)&uninitialized_var, uninitialized_var);
    printf("4. Heap Segment (Dynamic Allocation):\n");
    printf(" heap_var address: %p, value: %d\n\n",
        (void*)heap_var, *heap_var);

    printf("5. Stack Segment (Local Variables):\n");
    printf(" local_var address: %p, value: %d\n\n",
        (void*)&local_var, local_var);
    free(heap_var);
}
int main() {
    printf("Program dimulai...\n");
    printf("PID: %d\n\n", getpid());
    print_addresses();
    printf("\nTekan Enter untuk melihat memory map...");
    getchar();
    // Perintah untuk melihat memory map
    char cmd[100];
    sprintf(cmd, "cat /proc/%d/maps", getpid());
    system(cmd);
    return 0;
}
```

2. Contiguous Memory Allocation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h> #define MAX_BLOCKS 10
#define MEMORY_SIZE 1000
typedef struct {
    int id;
    int size;
    int start_address;
    int is_allocated;
} MemoryBlock;
MemoryBlock memory[MAX_BLOCKS];
int block_count = 0;
void initialize_memory() {
    // Inisialisasi dengan satu block kosong besar
    memory[0].id = -1; // -1 untuk free block
    memory[0].size = MEMORY_SIZE;
    memory[0].start_address = 0;
    memory[0].is_allocated = 0;
    block_count = 1;
}
void display_memory() {
    printf("\n=== STATUS MEMORI ===\n");
    printf("%-10s %-10s %-15s %-10s\n",
        "Block", "Size", "Start Address", "Status");
    printf("-----\n");
    for (int i = 0; i < block_count; i++) {
        printf("%-10d %-10d %-15d %-10s\n",
            memory[i].id,
            memory[i].size,
            memory[i].start_address,
            memory[i].is_allocated ? "Allocated" : "Free");
    }
    printf("\n");
}
int first_fit(int process_id, int size) {
    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated && memory[i].size >= size) {
            // Alokasi di block ini
            int old_size = memory[i].size;
            int old_start = memory[i].start_address;
            memory[i].id = process_id;
            memory[i].size = size;
            memory[i].is_allocated = 1;
            // Jika ada sisa, buat block baru
            if (old_size > size) {
                // Geser block-block setelahnya
                for (int j = block_count; j > i + 1; j--) {
                    memory[j] = memory[j-1];
                }
                memory[i+1].id = -1;
                memory[i+1].size = old_size - size;
                memory[i+1].start_address = old_start + size;
                memory[i+1].is_allocated = 0;
                block_count++;
            }
            printf("First-Fit: Process %d (%d bytes) dialokasikan di\n",
                address %d\n",
                process_id, size, old_start);
            return 1;
        }
    }
    printf("First-Fit: Gagal mengalokasikan Process %d (%d bytes)\n", process_id, size);
    return 0;
}
int best_fit(int process_id, int size) {
    int best_idx = -1;
    int smallest_hole = MEMORY_SIZE + 1;
    // Cari hole terkecil yang cukup
    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated && memory[i].size >= size) {
            if (memory[i].size < smallest_hole) {
```

```

        if (memory[i].size < smallest_hole) {
            smallest_hole = memory[i].size;
            best_idx = i;
        }
    }
}

if (best_idx == -1) {
    printf("Best-Fit: Gagal mengalokasikan Process %d (%d bytes)\n",
        process_id, size);
    return 0;
}

// Alokasi di best_idx
int old_size = memory[best_idx].size;
int old_start = memory[best_idx].start_address;
memory[best_idx].id = process_id;
memory[best_idx].size = size;
memory[best_idx].is_allocated = 1;
if (old_size > size) {
    for (int j = block_count; j > best_idx + 1; j--) {
        memory[j] = memory[j-1];
    }
    memory[best_idx+1].id = -1;
    memory[best_idx+1].size = old_size - size;
    memory[best_idx+1].start_address = old_start + size;
    memory[best_idx+1].is_allocated = 0;
    block_count++;
}
printf("Best-Fit: Process %d (%d bytes) dialokasikan di address %d\n",
    process_id, size, old_start);
return 1;
}

int worst_fit(int process_id, int size) {
    int worst_idx = -1;
    int largest_hole = -1;
    // Cari hole terbesar
    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated && memory[i].size >= size) {
            if (memory[i].size > largest_hole) {
                largest_hole = memory[i].size;
                worst_idx = i;
            }
        }
    }
    if (worst_idx == -1) {
        printf("Worst-Fit: Gagal mengalokasikan Process %d (%d bytes)\n",
            process_id, size);
        return 0;
    }
    // Alokasi di worst_idx
    int old_size = memory[worst_idx].size;
    int old_start = memory[worst_idx].start_address;
    memory[worst_idx].id = process_id;
    memory[worst_idx].size = size;
    memory[worst_idx].is_allocated = 1;
    if (old_size > size) {
        for (int j = block_count; j > worst_idx + 1; j--) {
            memory[j] = memory[j-1];
        }
        memory[worst_idx+1].id = -1;
        memory[worst_idx+1].size = old_size - size;
        memory[worst_idx+1].start_address = old_start + size;
        memory[worst_idx+1].is_allocated = 0;
        block_count++;
    }
    printf("Worst-Fit: Process %d (%d bytes) dialokasikan di address %d\n",
        process_id, size, old_start);
    return 1;
}

void deallocate(int process_id) {

```



```

for (int i = 0; i < block_count; i++) {
    if (memory[i].id == process_id && memory[i].is_allocated) {
        memory[i].id = -1;
        memory[i].is_allocated = 0;
        printf("Process %d berhasil di-dealokasi\n", process_id);
        // Merge dengan block sebelah jika free
        // Merge dengan block setelahnya
        if (i + 1 < block_count && !memory[i+1].is_allocated) {
            memory[i].size += memory[i+1].size;
            for (int j = i + 1; j < block_count - 1; j++) {
                memory[j] = memory[j+1];
            }
            block_count--;
        }
        // Merge dengan block sebelumnya
        if (i > 0 && !memory[i-1].is_allocated) {
            memory[i-1].size += memory[i].size;
            for (int j = i; j < block_count - 1; j++) {
                memory[j] = memory[j+1];
            }
            block_count--;
        }
        return;
    }
}
printf("Process %d tidak ditemukan\n", process_id);

}

void test_algorithm(int choice) {
    initialize_memory();
    printf("\n===== \n");
    printf("Testing: ");
    switch(choice) {
        case 1: printf("FIRST-FIT ALGORITHM\n"); break;
        case 2: printf("BEST-FIT ALGORITHM\n"); break;
        case 3: printf("WORST-FIT ALGORITHM\n"); break;
    }
    printf("===== \n");
    // Test case: alokasi beberapa process
    int processes[][2] = {
        {1, 100}, // Process 1: 100 bytes
        {2, 200}, // Process 2: 200 bytes
        {3, 300}, // Process 3: 300 bytes
        {4, 150}, // Process 4: 150 bytes
    };
    for (int i = 0; i < 4; i++) {
        switch(choice) {
            case 1: first_fit(processes[i][0], processes[i][1]); break;
            case 2: best_fit(processes[i][0], processes[i][1]); break;
            case 3: worst_fit(processes[i][0], processes[i][1]); break;
        }
    }
    display_memory();
    // Dealokasi process 2
    printf("Dealokasi Process 2... \n");
    deallocate(2);
    display_memory();
    // Alokasi process baru
    printf("Alokasi Process 5 (180 bytes)... \n");
    switch(choice) {
        case 1: first_fit(5, 180); break;
        case 2: best_fit(5, 180); break;
        case 3: worst_fit(5, 180); break;
    }
    display_memory();
}

int main() {
    int choice;
    while(1) {
        printf("\n=== SIMULASI CONTIGUOUS MEMORY ALLOCATION === \n");

```

3.

```

        printf("\n");
    }

```

```

int first_fit(int process_id, int size) {
    for (int i = 0; i < block_count; i++) {

```

```

        if (memory[i].size < smallest_hole) {
            smallest_hole = memory[i].size;
            best_idx = i;
        }
    }

    if (best_idx == -1) {
        printf("Best-Fit: Gagal mengalokasikan Process %d (%d bytes)\n",
            process_id, size);
        return 0;
    }

    // Alokasi di best_idx
    int old_size = memory[best_idx].size;
    int old_start = memory[best_idx].start_address;
    memory[best_idx].id = process_id;
    memory[best_idx].size = size;
    memory[best_idx].is_allocated = 1;
    if (old_size > size) {
        for (int j = block_count; j > best_idx + 1; j--) {
            memory[j] = memory[j-1];
        }
        memory[best_idx+1].id = -1;
        memory[best_idx+1].size = old_size - size;
        memory[best_idx+1].start_address = old_start + size;
        memory[best_idx+1].is_allocated = 0;
        block_count++;
    }
    printf("Best-Fit: Process %d (%d bytes) dialokasikan di address %d\n",
        process_id, size, old_start);
    return 1;
}

int worst_fit(int process_id, int size) {
    int worst_idx = -1;
    int largest_hole = -1;
    // Cari hole terbesar
    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated && memory[i].size >= size) {
            if (memory[i].size > largest_hole) {
                largest_hole = memory[i].size;
                worst_idx = i;
            }
        }
    }

    if (worst_idx == -1) {
        printf("Worst-Fit: Gagal mengalokasikan Process %d (%d bytes)\n",
            process_id, size);
        return 0;
    }

    // Alokasi di worst_idx
    int old_size = memory[worst_idx].size;
    int old_start = memory[worst_idx].start_address;
    memory[worst_idx].id = process_id;
    memory[worst_idx].size = size;
    memory[worst_idx].is_allocated = 1;
    if (old_size > size) {
        for (int j = block_count; j > worst_idx + 1; j--) {
            memory[j] = memory[j-1];
        }
        memory[worst_idx+1].id = -1;
        memory[worst_idx+1].size = old_size - size;
        memory[worst_idx+1].start_address = old_start + size;
        memory[worst_idx+1].is_allocated = 0;
        block_count++;
    }
    printf("Worst-Fit: Process %d (%d bytes) dialokasikan di address %d\n",
        process_id, size, old_start);
    return 1;
}

void deallocate(int process_id) {

```

```

for (int i = 0; i < block_count; i++) {
    if (memory[i].id == process_id && memory[i].is_allocated) {
        memory[i].id = -1;
        memory[i].is_allocated = 0;
        printf("Process %d berhasil di-dealokasi\n", process_id);
        // Merge dengan block sebelah jika free
        // Merge dengan block setelahnya
        if (i + 1 < block_count && !memory[i+1].is_allocated) {
            memory[i].size += memory[i+1].size;
            for (int j = i + 1; j < block_count - 1; j++) {
                memory[j] = memory[j+1];
            }
            block_count--;
        }
        // Merge dengan block sebelumnya
        if (i > 0 && !memory[i-1].is_allocated) {
            memory[i-1].size += memory[i].size;
            for (int j = i; j < block_count - 1; j++) {
                memory[j] = memory[j+1];
            }
            block_count--;
        }
        return;
    }
}
printf("Process %d tidak ditemukan\n", process_id);

}

void test_algorithm(int choice) {
    initialize_memory();
    printf("\n===== \n");
    printf("Testing: ");
    switch(choice) {
        case 1: printf("FIRST-FIT ALGORITHM\n"); break;
        case 2: printf("BEST-FIT ALGORITHM\n"); break;
        case 3: printf("WORST-FIT ALGORITHM\n"); break;
    }
    printf("===== \n");
    // Test case: alokasi beberapa process
    int processes[][2] = {
        {1, 100}, // Process 1: 100 bytes
        {2, 200}, // Process 2: 200 bytes
        {3, 300}, // Process 3: 300 bytes
        {4, 150}, // Process 4: 150 bytes
    };
    for (int i = 0; i < 4; i++) {
        switch(choice) {
            case 1: first_fit(processes[i][0], processes[i][1]); break;
            case 2: best_fit(processes[i][0], processes[i][1]); break;
            case 3: worst_fit(processes[i][0], processes[i][1]); break;
        }
    }
    display_memory();
    // Dealokasi process 2
    printf("Dealokasi Process 2...\n");
    deallocate(2);
    display_memory();
    // Alokasi process baru
    printf("Alokasi Process 5 (180 bytes)...\n");
    switch(choice) {
        case 1: first_fit(5, 180); break;
        case 2: best_fit(5, 180); break;
        case 3: worst_fit(5, 180); break;
    }
    display_memory();
}

int main() {
    int choice;
    while(1) {
        printf("\n=== SIMULASI CONTIGUOUS MEMORY ALLOCATION ===\n");

```

```

printf("1. First-Fit Algorithm\n");
printf("2. Best-Fit Algorithm\n");
printf("3. Worst-Fit Algorithm\n");
printf("4. Test Semua Algoritma\n");
printf("5. Exit\n");
printf("Pilihan: ");
scanf("%d", &choice);
if (choice == 5) break;
if (choice == 4) {
for (int i = 1; i <= 3; i++) {
test_algorithm(i);
printf("\nTekan Enter untuk lanjut...");
getchar(); getchar();
}
} else if (choice >= 1 && choice <= 3) {
test_algorithm(choice);
}
}
return 0;
}

```

```

printf("1. First-Fit Algorithm\n");
printf("2. Best-Fit Algorithm\n");
printf("3. Worst-Fit Algorithm\n");
printf("4. Test Semua Algoritma\n");
printf("5. Exit\n");
printf("Pilihan: ");
scanf("%d", &choice);
if (choice == 5) break;
if (choice == 4) {
for (int i = 1; i <= 3; i++) {
test_algorithm(i);
printf("\nTekan Enter untuk lanjut...");
getchar(); getchar();
}
} else if (choice >= 1 && choice <= 3) {
test_algorithm(choice);
}
}
return 0;
}

```

```

#include <stdio.h>
#include <stdlib.h>
int global_var = 100; // Data segment
int uninitialized_var; // BSS segment
void print_addresses() {
int local_var = 50; // Stack
int *heap_var = (int*)malloc(sizeof(int)); // Heap
*heap_var = 75;
printf("=== MEMORY ADDRESS LAYOUT ===\n\n");
printf("1. Text Segment (Code):\n");
printf(" Function address: %p\n", (void*)print_addresses);
printf("2. Data Segment (Initialized Global):\n");
printf(" global_var address: %p, value: %d\n",
(void*)&global_var, global_var);
printf("3. BSS Segment (Uninitialized Global):\n");
printf(" uninitialized_var address: %p, value: %d\n",
(void*)&uninitialized_var, uninitialized_var);
printf("4. Heap Segment (Dynamic Allocation):\n");
printf(" heap_var address: %p, value: %d\n",
(void*)heap_var, *heap_var);
printf("5. Stack Segment (Local Variables):\n");

```

```

for (int i = 0; i < block_count; i++) {
    if (memory[i].id == process_id && memory[i].is_allocated) {
        memory[i].id = -1;
        memory[i].is_allocated = 0;
        printf("Process %d berhasil di-dealokasi\n", process_id);
        // Merge dengan block sebelah jika free
        // Merge dengan block setelahnya
        if (i + 1 < block_count && !memory[i+1].is_allocated) {
            memory[i].size += memory[i+1].size;
            for (int j = i + 1; j < block_count - 1; j++) {
                memory[j] = memory[j+1];
            }
            block_count--;
        }
        // Merge dengan block sebelumnya
        if (i > 0 && !memory[i-1].is_allocated) {
            memory[i-1].size += memory[i].size;
            for (int j = i; j < block_count - 1; j++) {
                memory[j] = memory[j+1];
            }
            block_count--;
        }
        return;
    }
}
printf("Process %d tidak ditemukan\n", process_id);

}

void test_algorithm(int choice) {
    initialize_memory();
    printf("\n===== \n");
    printf("Testing: ");
    switch(choice) {
        case 1: printf("FIRST-FIT ALGORITHM\n"); break;
        case 2: printf("BEST-FIT ALGORITHM\n"); break;
        case 3: printf("WORST-FIT ALGORITHM\n"); break;
    }
    printf("===== \n");
    // Test case: alokasi beberapa process
    int processes[][2] = {
        {1, 100}, // Process 1: 100 bytes
        {2, 200}, // Process 2: 200 bytes
        {3, 300}, // Process 3: 300 bytes
        {4, 150}, // Process 4: 150 bytes
    };
    for (int i = 0; i < 4; i++) {
        switch(choice) {
            case 1: first_fit(processes[i][0], processes[i][1]); break;
            case 2: best_fit(processes[i][0], processes[i][1]); break;
            case 3: worst_fit(processes[i][0], processes[i][1]); break;
        }
    }
    display_memory();
    // Dealokasi process 2
    printf("Dealokasi Process 2...\n");
    deallocate(2);
    display_memory();
    // Alokasi process baru
    printf("Alokasi Process 5 (180 bytes)...\n");
    switch(choice) {
        case 1: first_fit(5, 180); break;
        case 2: best_fit(5, 180); break;
        case 3: worst_fit(5, 180); break;
    }
    display_memory();
}

int main() {
    int choice;
    while(1) {
        printf("\n=== SIMULASI CONTIGUOUS MEMORY ALLOCATION ===\n");

        if (choice == 4) {
            for (int i = 1; i <= 3; i++) {
                test_algorithm(i);
                printf("\nTekan Enter untuk lanjut...");
                getchar(); getchar();
            }

```

```

for (int i = 0; i < block_count; i++) {
    if (memory[i].id == process_id && memory[i].is_allocated) {
        memory[i].id = -1;
        memory[i].is_allocated = 0;
        printf("Process %d berhasil di-dealokasi\n", process_id);
        // Merge dengan block sebelah jika free
        // Merge dengan block setelahnya
        if (i + 1 < block_count && !memory[i+1].is_allocated) {
            memory[i].size += memory[i+1].size;
            for (int j = i + 1; j < block_count - 1; j++) {
                memory[j] = memory[j+1];
            }
            block_count--;
        }
        // Merge dengan block sebelumnya
        if (i > 0 && !memory[i-1].is_allocated) {
            memory[i-1].size += memory[i].size;
            for (int j = i; j < block_count - 1; j++) {
                memory[j] = memory[j+1];
            }
            block_count--;
        }
        return;
    }
}
printf("Process %d tidak ditemukan\n", process_id);

}

void test_algorithm(int choice) {
    initialize_memory();
    printf("\n===== \n");
    printf("Testing: ");
    switch(choice) {
        case 1: printf("FIRST-FIT ALGORITHM\n"); break;
        case 2: printf("BEST-FIT ALGORITHM\n"); break;
        case 3: printf("WORST-FIT ALGORITHM\n"); break;
    }
    printf("===== \n");
    // Test case: alokasi beberapa process
    int processes[][2] = {
        {1, 100}, // Process 1: 100 bytes
        {2, 200}, // Process 2: 200 bytes
        {3, 300}, // Process 3: 300 bytes
        {4, 150}, // Process 4: 150 bytes
    };
    for (int i = 0; i < 4; i++) {
        switch(choice) {
            case 1: first_fit(processes[i][0], processes[i][1]); break;
            case 2: best_fit(processes[i][0], processes[i][1]); break;
            case 3: worst_fit(processes[i][0], processes[i][1]); break;
        }
    }
    display_memory();
    // Dealokasi process 2
    printf("Dealokasi Process 2...\n");
    deallocate(2);
    display_memory();
    // Alokasi process baru
    printf("Alokasi Process 5 (180 bytes)...\n");
    switch(choice) {
        case 1: first_fit(5, 180); break;
        case 2: best_fit(5, 180); break;
        case 3: worst_fit(5, 180); break;
    }
    display_memory();
}

int main() {

```

```

for (int i = 0; i < block_count; i++) {
    if (memory[i].id == process_id && memory[i].is_allocated) {
        memory[i].id = -1;
        memory[i].is_allocated = 0;
        printf("Process %d berhasil di-dealokasi\n", process_id);
        // Merge dengan block sebelah jika free
        // Merge dengan block setelahnya
        if (i + 1 < block_count && !memory[i+1].is_allocated) {
            memory[i].size += memory[i+1].size;
            for (int j = i + 1; j < block_count - 1; j++) {
                memory[j] = memory[j+1];
            }
            block_count--;
        }
        // Merge dengan block sebelumnya
        if (i > 0 && !memory[i-1].is_allocated) {
            memory[i-1].size += memory[i].size;
            for (int j = i; j < block_count - 1; j++) {
                memory[j] = memory[j+1];
            }
            block_count--;
        }
        return;
    }
}
printf("Process %d tidak ditemukan\n", process_id);

}

void test_algorithm(int choice) {
    initialize_memory();
    printf("\n===== \n");
    printf("Testing: ");
    switch(choice) {
        case 1: printf("FIRST-FIT ALGORITHM\n"); break;
        case 2: printf("BEST-FIT ALGORITHM\n"); break;
        case 3: printf("WORST-FIT ALGORITHM\n"); break;
    }
    printf("===== \n");
    // Test case: alokasi beberapa process
    int processes[][2] = {
        {1, 100}, // Process 1: 100 bytes
        {2, 200}, // Process 2: 200 bytes
        {3, 300}, // Process 3: 300 bytes
        {4, 150}, // Process 4: 150 bytes
    };
    for (int i = 0; i < 4; i++) {
        switch(choice) {
            case 1: first_fit(processes[i][0], processes[i][1]); break;
            case 2: best_fit(processes[i][0], processes[i][1]); break;
            case 3: worst_fit(processes[i][0], processes[i][1]); break;
        }
    }
    display_memory();
    // Dealokasi process 2
    printf("Dealokasi Process 2... \n");
    deallocate(2);
    display_memory();
    // Alokasi process baru
    printf("Alokasi Process 5 (180 bytes)... \n");
    switch(choice) {
        case 1: first_fit(5, 180); break;
        case 2: best_fit(5, 180); break;
    }
}

```

3.3. Screenshot output

```
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_7$ gcc -o adrrs address_demo.c
address_demo.c: In function 'main':
address_demo.c:29:27: warning: implicit declaration of function 'getpid' [-Wimplicit-function-declaration]
   29 |     printf("PID: %d\n\n", getpid());
      |                          ~~~~~
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_7$ █

wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_7$ gcc -o adrrs address_demo.c
address_demo.c: In function 'main':
address_demo.c:29:27: warning: implicit declaration of function 'getpid' [-Wimplicit-function-declaration]
   29 |     printf("PID: %d\n\n", getpid());
      |                          ~~~~~
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_7$ █
```

3.4. Pejelasan

BAB 4

ANALISIS DAN PEMBAHASAN

BAB 5

DAFTAR PUSTAKA